

Φάση Δ: Advanced Technique

Εμείς επιλέξαμε το K-Means

```
# Φάση Δ: Advanced Technique
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.ml.fpm import FPGrowth
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import PCA

spark = SparkSession.builder \
    .appName("Advanced_Techniques") \
    .master("local[*]") \
    .config("spark.driver.memory", "4g") \
    .config("spark.executor.memory", "4g") \
    .getOrCreate()

# --- 1. ASSOCIATION RULES ---
print("Εξαγωγή Κανόνων Συσχέτισης...")
df_silver = spark.read.parquet("../data/train_silver.parquet")

transactions_df = df_silver.withColumn(
    "items",
    F.array_remove(F.array(
        F.when(F.col("stroke") == 1, "Stroke").otherwise(""),
        F.when(F.col("hypertension") == 1, "Hypertension").otherwise(""),
        F.when(F.col("heart_disease") == 1, "Heart_Disease").otherwise(""),
        F.when(F.col("age") > 65, "Age_>_65").otherwise("")
    ), "")
).filter(F.size("items") > 0).select("items")

fpGrowth = FPGrowth(itemsCol="items", minSupport=0.01, minConfidence=0.1)
rules = fpGrowth.fit(transactions_df).associationRules

print("Κανόνες Κινδύνου για Εγκεφαλικό:")
rules.filter(F.array_contains(F.col("consequent"),
    "Stroke")).sort(F.col("lift").desc()).show(truncate=False)

# --- 2. CLUSTERING (K-MEANS) ---
print("Εκτέλεση K-Means...")
train_gold = spark.read.parquet("../data/train_gold.parquet")
kmeans = KMeans(featuresCol="features", predictionCol="cluster", k=2,
    seed=22390225)
#kmeans = KMeans(featuresCol="features", predictionCol="cluster", seed=22390225)

kmeans_preds = kmeans.fit(train_gold).transform(train_gold)

pca = PCA(k=2, inputCol="features", outputCol="pca_features")
```

```
kmeans_pca = pca.fit(kmeans_preds).transform(kmeans_preds)

kmeans_pca.select("stroke", "cluster",
"pca_features").write.mode("overwrite").parquet("../data/preds_kmeans.parquet")
#spark.stop()
```

Εξαγωγή Κανόνων Συσχέτισης...

Κανόνες Κινδύνου για Εγκεφαλικό:

```
+-----+-----+-----+-----+
+-----+
|antecedent          |consequent|confidence      |lift            |
support              |          |                 |                |
+-----+-----+-----+-----+
+-----+
|[Hypertension, Age_>_65] |[Stroke]  |0.23076923076923078|1.2396878483835005|
0.03776978417266187 |
|[Heart_Disease, Age_>_65] |[Stroke]  |0.21014492753623187|1.128894489953091 |
0.026079136690647483|
|[Heart_Disease]          |[Stroke]  |0.18018018018018017|0.9679244461853157|
0.03597122302158273 |
|[Age_>_65]              |[Stroke]  |0.1671018276762402 |0.8976677892559377|
0.11510791366906475 |
|[Hypertension]          |[Stroke]  |0.14039408866995073|0.7541943314057257|
0.051258992805755396|
+-----+-----+-----+-----+
+-----+
```

Εκτέλεση K-Means...

```
26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers
26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers
26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 76.00% for 10 writers
26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 69.09% for 11 writers
26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 76.00% for 10 writers
```

```

26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/06/05 18:39:42 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers

# K Means μέσω Elbow Method
# Kmeans μέσω elbow method
import matplotlib.pyplot as plt
import pandas as pd

print("Εύρεση βέλτιστου K με την Elbow Method...")

# Διαβάζουμε τα χρυσά δεδομένα (αν δεν είναι ήδη φορτωμένα)
train_gold = spark.read.parquet("../data/train_gold.parquet")

# Λίστα για την αποθήκευση του κόστους (Within Set Sum of Squared Errors)
costs = []
k_range = range(2, 9) # Δοκιμάζουμε k από 2 έως 8

for k_val in k_range:
    # Εκπαίδευση του μοντέλου για το τρέχον k
    km = KMeans(featuresCol="features", k=k_val, seed=42)
    model = km.fit(train_gold)

    # Στο PySpark, το trainingCost μας δίνει το WSSSE
    cost = model.summary.trainingCost
    costs.append(cost)
    print(f"Για K = {k_val}, το Training Cost (WSSSE) είναι: {cost:.2f}")

# Δημιουργία γραφήματος για την οπτικοποίηση του "αγκώνα"
plt.figure(figsize=(8, 5))
plt.plot(list(k_range), costs, marker='o', linestyle='-', color='b')
plt.xlabel('Αριθμός Clusters (K)')
plt.ylabel('Training Cost (WSSSE)')
plt.title('Elbow Method για την εύρεση του βέλτιστου K')
plt.grid(True)
plt.show()

# --- Σημείωση: Αφού δεις το γράφημα, επιλέγεις το K εκεί που "σπάει" η καμπύλη
---
```

Εύρεση βέλτιστου K με την Elbow Method...

Για K = 2, το Training Cost (WSSSE) είναι: 143070.51

Για K = 3, το Training Cost (WSSSE) είναι: 130855.85

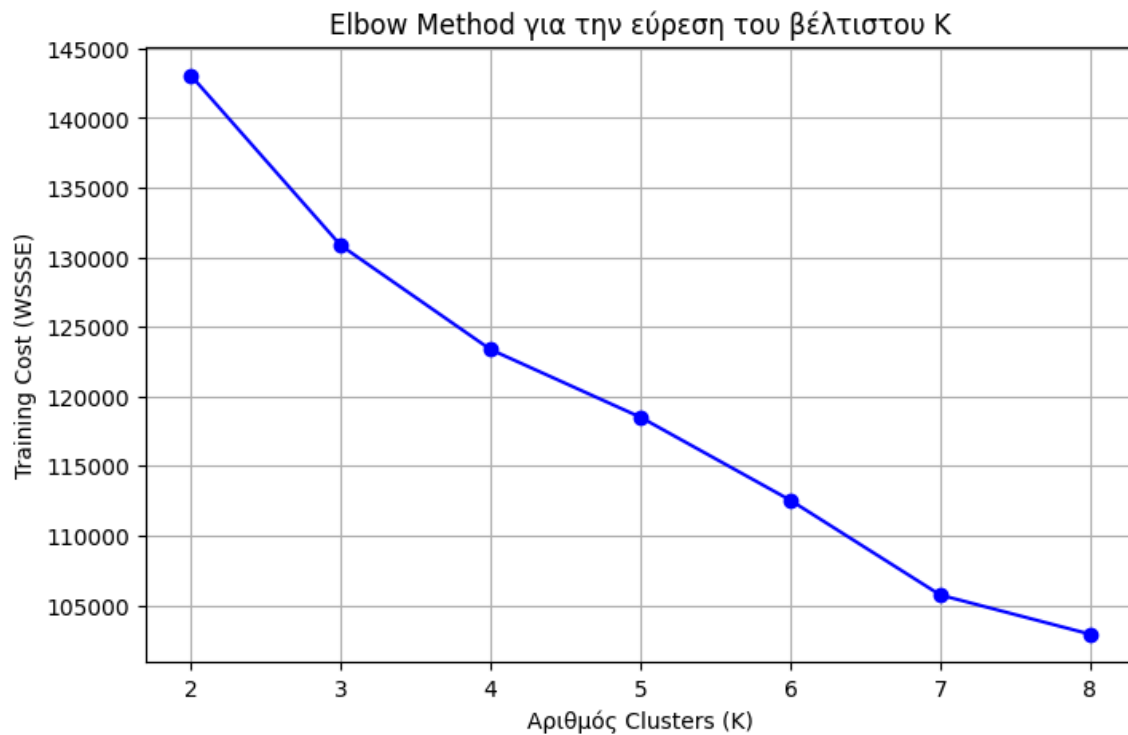
Για K = 4, το Training Cost (WSSSE) είναι: 123369.98

Για K = 5, το Training Cost (WSSSE) είναι: 118505.81

Για K = 6, το Training Cost (WSSSE) είναι: 112539.68

Για K = 7, το Training Cost (WSSSE) είναι: 105731.45

Για K = 8, το Training Cost (WSSSE) είναι: 102912.18



```
# K Means μέσω Silhouette Score
# K Means μέσω Silhouette Score
import matplotlib.pyplot as plt
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

print("Εύρεση βέλτιστου K με το Silhouette Score...")

# Υποθέτουμε ότι το train_gold υπάρχει ήδη (π.χ. train_gold =
spark.read.parquet("../data/train_gold.parquet"))

silhouette_scores = []
k_range = range(2, 9) # Δοκιμάζουμε K από 2 έως 8

# Ορίζουμε τον Αξιολογητή (Evaluator)
# Σημείωση: Χρησιμοποιούμε predictionCol="cluster" όπως το είχες ορίσει αρχικά
evaluator = ClusteringEvaluator(predictionCol="cluster", featuresCol="features",
metricName="silhouette")

for k_val in k_range:
    # 1. Ορισμός και εκπαίδευση του μοντέλου
    km = KMeans(featuresCol="features", predictionCol="cluster", k=k_val,
seed=42)
```

```

model = km.fit(train_gold)

# 2. Παραγωγή προβλέψεων (τα assignments στα clusters)
predictions = model.transform(train_gold)

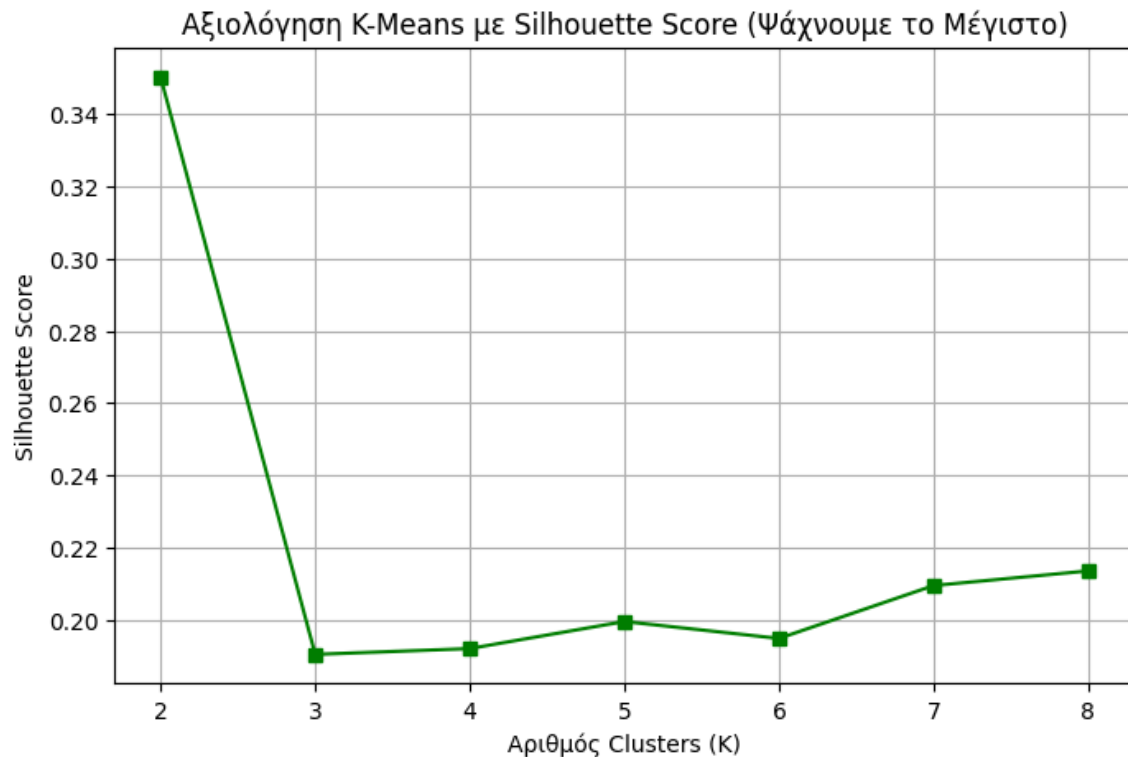
# 3. Υπολογισμός του Silhouette Score
score = evaluator.evaluate(predictions)
silhouette_scores.append(score)
print(f"Για K = {k_val}, το Silhouette Score είναι: {score:.4f}")

# Δημιουργία γραφήματος για την οπτικοποίηση των σκορ
plt.figure(figsize=(8, 5))
plt.plot(list(k_range), silhouette_scores, marker='s', linestyle='--', color='g')
plt.xlabel('Αριθμός Clusters (K)')
plt.ylabel('Silhouette Score')
plt.title('Αξιολόγηση K-Means με Silhouette Score (Ψάχνουμε το Μέγιστο)')
plt.grid(True)
plt.show()

# --- Σημείωση: Επίλεξε το K που αντιστοιχεί στο υψηλότερο σημείο του γραφήματος
---
```

Εύρεση βέλτιστου K με το Silhouette Score...

K	Silhouette Score
2	0.3502
3	0.1905
4	0.1921
5	0.1996
6	0.1949
7	0.2095
8	0.2136



```
import matplotlib.pyplot as plt
import pandas as pd

print("Προετοιμασία δεδομένων για το Plot των K-Means Clusters...")
# Προσοχή: Εδώ επιλέγουμε τη στήλη 'cluster', ΟΧΙ το 'stroke'
plot_df = kmeans_pca.select("pca_features", "cluster").toPandas()

# Εξαγωγή των PC1 και PC2
plot_df['PC1'] = plot_df['pca_features'].apply(lambda vec: float(vec[0]))
plot_df['PC2'] = plot_df['pca_features'].apply(lambda vec: float(vec[1]))

plt.figure(figsize=(10, 6))

# Ορίζουμε 3 χρώματα για τα 3 clusters (0, 1, 2)
colors = {0: 'blue', 1: 'green', 2: 'red'}

# Φτιάχνουμε τα σημεία για κάθε cluster
for cluster_id in sorted(plot_df['cluster'].unique()):
    cluster_data = plot_df[plot_df['cluster'] == cluster_id]
    plt.scatter(cluster_data['PC1'], cluster_data['PC2'],
                alpha=0.6, label=f'Cluster {cluster_id}',
                color=colors.get(cluster_id, 'black'), s=30)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
```

```
plt.title(f'Οπτικοποίηση των K-Means Clusters (K={kmeans.getK()})')  
plt.legend()  
plt.grid(True)  
plt.show()
```

Προετοιμασία δεδομένων για το Plot των K-Means Clusters...

